

# Introduzione ai paradigmi di programmazione e breve storia dei linguaggi funzionali

Luca Padovani

Linguaggi e Paradigmi di Programmazione



Questo documento è distribuito con licenza CC BY-NC-SA 4.0. Generato il 18 agosto 2020.

# paradigmi di programmazione

| Paradigma       | Un programma è                                                  | Esempi    |
|-----------------|-----------------------------------------------------------------|-----------|
| Imperativo      | sequenza di azioni<br>esecuzione $\Rightarrow$ nuovo stato      | C, Pascal |
| Object-oriented | oggetti che comunicano<br>interazione $\Rightarrow$ nuovo stato | Smalltalk |
| Funzionale      | espressione<br>valutazione $\Rightarrow$ risultato              | Haskell   |

- ▶ quasi tutti i linguaggi in uso sono **multi-paradigma**

# paradigmi di programmazione

| Paradigma       | Un programma è                                                  | Esempi                                            |
|-----------------|-----------------------------------------------------------------|---------------------------------------------------|
| Imperativo      | sequenza di azioni<br>esecuzione $\Rightarrow$ nuovo stato      | C, Pascal, C++,<br>Java, Scala,<br>Python, Scheme |
| Object-oriented | oggetti che comunicano<br>interazione $\Rightarrow$ nuovo stato | Smalltalk, C++,<br>OCaml, Java, Scala,<br>Python  |
| Funzionale      | espressione<br>valutazione $\Rightarrow$ risultato              | Haskell, OCaml,<br>Scala, Erlang,<br>Scheme       |

- ▶ quasi tutti i linguaggi in uso sono **multi-paradigma**

# storia (parziale) dei linguaggi di programmazione

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| '50 | <p><b>Assembly</b>: corrispondenza 1-1 con codice macchina, mancano visioni su linguaggi ad alto livello, si teme che non sarà mai possibile scrivere programmi complessi</p> <p><b>FORTRAN</b>: linguaggio per <b>calcoli numerici</b>, introduce <b>procedure</b> (no ricorsione), cicli, <b>espressioni</b> in notazione infissa, I/O formattato, usato ancora oggi (con raffinamenti), richiede 10 anni/uomo di sviluppo, bootstrap difficilissimo (gli studi contemporanei di Chomsky furono applicati solo in seguito)</p> <p><b>COBOL</b>: linguaggio per gestione aziendale, introduce il <b>record</b></p> |
| '60 | <p><b>ALGOL</b>: grammatica in forma <b>BNF</b> (Backus-Naur Form), primo linguaggio indipendente dall'architettura, introduce <b>blocchi</b> con <b>variabili locali</b>, <b>ricorsione</b></p> <p><b>BASIC</b>: "programmazione per tutti", sarà incorporato nei primi Personal Computer (IBM e Apple) 20 anni dopo, facile da imparare ma poco strutturato (goto)</p> <p><b>Simula 67</b>: astrazioni di più alto livello, introduce <b>classi</b>, <b>oggetti</b> e <b>ereditarietà</b></p>                                                                                                                     |
| '70 | <p><b>Pascal</b>: programmazione <b>strutturata</b>, ruolo importante dei <b>tipi</b> per evitare errori di programmazione, ideale per imparare a programmare</p> <p><b>Smalltalk</b>: programmazione a <b>oggetti</b> estrema, tutto è un oggetto (numeri, classi, ecc.), l'unica operazione possibile è l'invio di un <b>messaggio</b> (metodo)</p> <p><b>C</b>: linguaggio di medio livello per facilitare l'accesso all'hardware e implementazione di sistemi operativi (UNIX)</p>                                                                                                                              |

# storia (parziale) dei linguaggi di programmazione

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| '80 | <p><b>Ada</b>: sviluppato dal ministero della difesa USA, evoluzione del Pascal con <b>concorrenza</b> e <b>tipi astratti</b> (rappresentazione privata + interfaccia pubblica)</p> <p><b>C++</b>: estensione di C con classi e oggetti, introduce i <b>template</b> (classi generiche ottenute per espansione) e <b>overloading</b> di operatori e metodi</p> <p><b>PostScript</b>: linguaggio stack-based creato da Adobe per descrivere documenti, interprete in stampanti, verrà semplificato e reso efficiente in PDF</p> <p><b>Perl</b>: prestazioni PC rendono possibile uso di linguaggi <b>interpretati</b>, Perl è per "sistemisti", manipolazione di file di configurazione del S.O., find/replace con <b>espressioni regolari</b>, linguaggio <b>write-only</b> (programmi difficili da leggere)</p> |
| '90 | <p><b>Python</b>: linguaggio di scripting "della domenica", versione a oggetti di Perl</p> <p><b>Java</b>: versione ripulita e robusta di C++ per la programmazione con classi e oggetti, idea "compile-once-run-everywhere" grazie alla <b>JVM</b>, pensato per <b>dispositivi mobili</b>, sistemi embedded (lavatrici) e Web (<b>applet</b>), diventerà mainstream</p> <p><b>PHP</b>: produzione di pagine <b>Web</b> da database</p> <p><b>JavaScript</b>: produzione di <b>pagine Web interattive</b></p>                                                                                                                                                                                                                                                                                                    |
| '00 | <p><b>C#</b>: vuole essere C++ "fatto bene", compilatore per la piattaforma .NET di Microsoft e successivamente altri S.O. (mono), contributi da un comitato di esperti</p> <p><b>Scala</b>: linguaggio a oggetti con sintassi "flessibile", incorpora caratteristiche da linguaggi funzionali, compilatore per JVM</p> <p><b>Go</b>: linguaggio di Google per programmazione <b>concorrente</b> e <b>distribuita</b> su <b>scala globale</b>, comunicazione su <b>canali</b>, elimina <b>classi</b> a favore delle <b>interfacce</b></p>                                                                                                                                                                                                                                                                        |

nel frattempo...

- ▶ Alonzo Church (1903–1995)
- ▶ “calcolare” con le funzioni, così come si calcola con i numeri
- ▶ tutto è una funzione (numeri, liste, costrutti di controllo, ecc.)
- ▶ funzioni **anonime**. E.g. identità

$\lambda x.x$

- ▶ funzioni a un argomento, **currying** (da Haskell Curry)
- ▶ funzioni di **ordine superiore** (funzioni applicate a funzioni)
- ▶ il  $\lambda$  nasce da errore tipografico



## Fattoriale

```

$$\lambda n.n \ (\lambda f.\lambda a.\lambda x.f \ ((\lambda x.\lambda y.\lambda z.x \ (y \ z)) \ a \ x) \\ ((\lambda n.\lambda f.\lambda x.f \ (n \ f \ x)) \ x)) \ (\lambda x.\lambda y.x) \\ (\lambda f.\lambda x.f \ x) \ (\lambda f.\lambda x.f \ x)$$

```

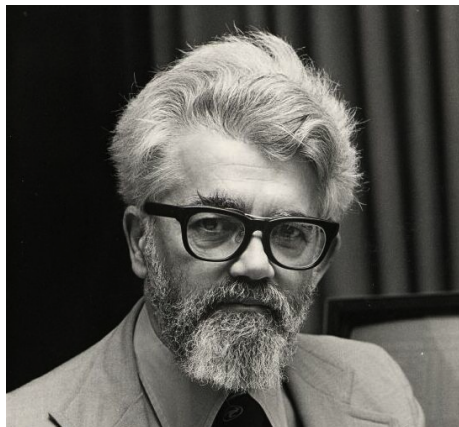
## Concatenazione di due liste

```

$$(\lambda g.(\lambda x.g \ (x \ x)) \ (\lambda x.g \ (x \ x))) \\ (\lambda g.\lambda c.\lambda x.(\lambda p.p \ (\lambda x.\lambda y.\lambda x.\lambda y.y)) \ x \ c \\ (g \ ((\lambda n.\lambda f.\lambda x.f \ (n \ f \ x)) \ c) \ ((\lambda p.p \ \lambda x.\lambda y.y) \ x))) \\ (\lambda f.\lambda x.x)$$

```

- ▶ John McCarthy (1927–2011)
- ▶ elaborazione informazione non-numerica/simbolica, i programmi sono (anche) dati
- ▶ ruolo chiave delle **liste**
  - LISP = LISt Processor
  - “cons” e “map” nascono qui
- ▶ primo **garbage collector**
- ▶ **funzioni anonime** ( $\lambda$  calcolo)
- ▶ notazione prefissa (facile implementare parser)
- ▶ **impuro**, forte legame con architettura di Von Neumann



## Fattoriale

```
(defun factorial (n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))
```

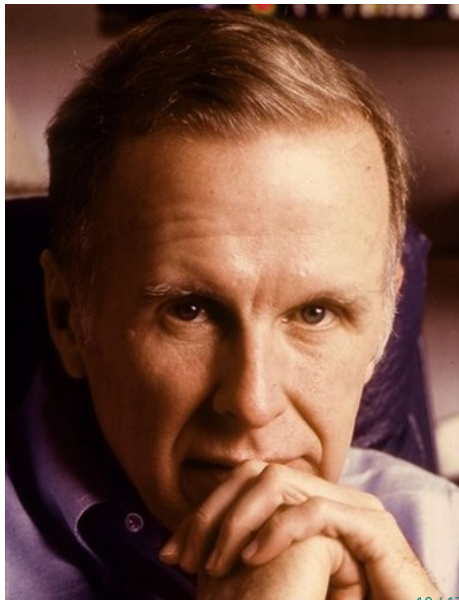
## Concatenazione di due liste

```
(defun append (l1 l2)
  (if (null l1)
      l2
      (cons (first l1) (append (rest l1) l2))))
```

- ▶ John Backus (1924–2007)
- ▶ vince il premio Turing per il **FORTRAN** nel 1977 ma durante la lezione d'onore critica il paradigma imperativo ("word-at-a-time" programming) e introduce **FP**
- ▶ accende l'interesse per i linguaggi funzionali, anche se FP non avrà mai successo
- ▶ programmi in stile **point-free**

$$(/+) \circ (\alpha \times) \circ \textit{Transpose}$$

prodotto scalare di 2 vettori



- ▶ Robin Milner (1934–2010)
- ▶ “spin-off” di LCF (Logic for Computable Functions), diventa linguaggio a sé stante
- ▶ linguaggio fortemente tipato (no errori di tipo a runtime)
- ▶ **type inference** (tipi inferiti dal compilatore)
- ▶ **polimorfismo parametrico** (30 anni prima dei Java generics)
- ▶ **tipi** definibili dal programmatore
- ▶ **pattern matching**
- ▶ sistema di **moduli**



## Fattoriale

```
fun factorial n =  
  if n = 0 then 1  
  else n * factorial (n - 1)
```

## Concatenazione di due liste

```
fun append (xs, ys) =  
  case xs of  
    [] => ys  
  | (hd :: tl) => hd :: append (tl, ys)
```

- ▶ David Turner e altri
- ▶ linguaggi **lazy**, con valutazione "on demand" degli argomenti
- ▶ SASL introduce **guardie** e **currying**
- ▶ KRC introduce il costrutto precursore delle **list comprehension**

$$[x^2 \mid x \leftarrow [1..100], \text{odd}(x)]$$

- ▶ Miranda introduce le **sezioni**

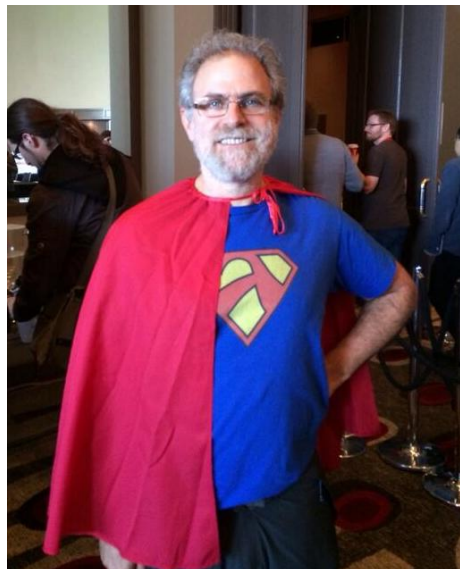
(/ 2)



## Fattoriale con guardie

```
fac n = 1,          n = 0  
      = n * fac (n - 1),  n > 0
```

- ▶ Phil Wadler e molti altri (comitato scientifico)
- ▶ linguaggio **lazy** “standardizzato”
- ▶ separazione tra parte pura e parte impura per mezzo di **monadi** (I/O, stato, ...)
- ▶ overloading con **type classes**
- ▶ laboratorio di ricerca per linguaggi lazy e **sistemi di tipo** avanzati
- ▶ seconda vita dopo il 2005 per impatto su **calcolo parallelo**



## Fattoriale

```
factorial n | n == 0    = 1  
           | otherwise = n * factorial (n - 1)
```

## Concatenazione di due liste

```
append []      ys = ys  
append (x : xs) ys = x : append xs ys
```

